You said:

complete set basics to advanced

ChatGPT said:

Python Set — Complete Guide (Basics to Advanced)

This guide covers Python sets from beginner level to advanced concepts, with simple explanations, practical examples, dry runs, common mistakes, coding interview questions, and practice exercises.

1. What is a set in Python?

A **set** is a built-in Python data type that stores a collection of **unique elements**.

Think of a set as a basket where duplicate items are automatically removed.

Example: Removing duplicate values

Original list	→	Unique values
[10, 20, 10, 30, 20]		{10, 20, 30}

Python keeps only one occurrence of each value. The order in which elements appear when displayed is not guaranteed.

```
numbers = {10, 20, 10, 30, 20}

print(numbers)
print(len(numbers))
```

Possible output:

```
{10, 20, 30}
3
```

The displayed order may vary.

Important properties of a set

- **Unique:** duplicates are automatically removed.
- **Unordered:** elements have no guaranteed positional order.
- **Mutable:** you can add or remove elements.
- **Unindexed:** you cannot access an element using `my_set[0]`.
- **Hashable elements:** elements must be hashable, such as integers, strings, and tuples containing only hashable values.
- **Mixed data types:** a set can contain values of different types.

2. How to create a set

A. Using curly braces `{}`

```
numbers = {1, 2, 3, 4, 5}
fruits = {"apple", "banana", "mango"}

print(numbers)
print(fruits)
```

B. Using `set()`

```
numbers = set([1, 2, 2, 3, 4])
print(numbers)
```

Output:

```
{1, 2, 3, 4}
```

You can convert other iterables into sets to remove duplicates.

```
text = set("hello")
print(text)
```

This produces a set of unique characters; their display order can vary.

C. Creating an empty set

This is a very common interview question.

```
a = set()
b = {}

print(type(a))
print(type(b))
```

Output:

```
<class 'set'>
<class 'dict'>
```

Remember: `{}` creates an empty dictionary, not an empty set. Use `set()` for an empty set.

3. Accessing and checking set elements

Because sets are unordered and unindexed, you cannot use indexing or slicing.

```
colors = {"red", "green", "blue"}

# Membership testing
print("red" in colors)
print("yellow" in colors)
```

Output:

```
True
False
```

Loop through a set

```
numbers = {10, 20, 30}

for number in numbers:
    print(number)
```

The elements are visited, but you should not rely on a particular order.

Dry run

For this code:

```
numbers = {10, 20, 30}
```

```
for n in numbers:
    print(n * 2)
```

Current element (`n`)	Calculation	Result
10	10 × 2	20
20	20 × 2	40
30	30 × 2	60

The order of output may differ.

4. Adding elements to a set

Python provides two important methods: `add()` and `update()`.

A. `add()` — add one element

Syntax:

```
my_set.add(element)
```

Example:

```
numbers = {10, 20, 30}

numbers.add(40)
numbers.add(20)

print(numbers)
```

The set contains `40`, but adding `20` again does not create a duplicate.

B. `update()` — add multiple elements

```
numbers = {1, 2, 3}

numbers.update([4, 5, 6])

print(numbers)
```

You can pass an iterable such as a list, tuple, string, or another set.

```
a = {1, 2}
a.update((3, 4))
a.update({5, 6})

print(a)
```

Result: a set containing `1, 2, 3, 4, 5, 6`.

`add()` vs `update()`

Feature	`add()`	`update()`
Purpose	Adds one element	Adds elements from an iterable
Example	`s.add(10)`	`s.update([10, 20])`
Argument	One element	One or more iterables
Returns	`None`	`None`

Important: `s.add([1, 2])` raises a `TypeError` because a list is unhashable. Use `s.update([1, 2])` to add its individual elements.

5. Removing elements from a set

Python has four commonly used methods.

A. `remove()`

Removes the specified element. Raises `KeyError` if the element does not exist.

```
numbers = {10, 20, 30}
numbers.remove(20)

print(numbers)
```

B. `discard()`

Removes an element if present. Does not raise an error if it is missing.

```
numbers = {10, 20, 30}
numbers.discard(100)

print(numbers)
```

The set remains unchanged.

C. `pop()`

Removes and returns an arbitrary element. You cannot choose which element is removed.

```
numbers = {10, 20, 30}

removed = numbers.pop()

print("Removed:", removed)
print("Remaining:", numbers)
```

The removed element may vary.

D. `clear()`

Removes every element while keeping the set object itself.

```
numbers = {10, 20, 30}
numbers.clear()

print(numbers)
```

Output: `set()`.

Comparison table

Method	If element is absent	Return value
`remove(x)`	Raises `KeyError`	`None`
`discard(x)`	No error	`None`
`pop()`	Raises `KeyError` if empty	Removed element
`clear()`	No error	`None`

Interview tip: When you are not sure whether an element exists, `discard()` is often safer than `remove()`.

6. Set operations — the most important topic

Set operations let you compare two groups of data. They are especially useful for removing duplicates, comparing student enrollments, and finding common interests.

Suppose:

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

Visual guide to set operations



Elements 3 and 4 are shared by both sets.

A. Union — `A | B`

Returns all unique elements from both sets.

```
print(A | B)
print(A.union(B))
```

Result: `{1, 2, 3, 4, 5, 6}`

Real-world example: students enrolled in either course.

B. Intersection — `A & B`

Returns only common elements.

```
print(A & B)
print(A.intersection(B))
```

Result: `{3, 4}`

Real-world example: students enrolled in both courses.

C. Difference — `A - B`

Returns elements present in `A` but not in `B`.

```
print(A - B)
print(A.difference(B))
```

Result: `{1, 2}`

Reverse the operands and you get the opposite difference:

```
print(B - A)
```

Result: `{5, 6}`

Remember: Difference is directional. `A - B` is not necessarily equal to `B - A`.

D. Symmetric difference — `A ^ B`

Returns elements present in either set, but not in both.

```
print(A ^ B)
print(A.symmetric_difference(B))
```

Result: `{1, 2, 5, 6}`

Real-world example: students enrolled in exactly one of two courses.

Quick revision

Operation	Operator	Meaning
Union	`A | B`	All unique elements
Intersection	`A & B`	Common elements
Difference	`A - B`	In A, not B
Symmetric difference	`A ^ B`	In exactly one set

Try the interactive quiz:

Question: Let `A = {1, 2, 3}` and `B = {3, 4, 5}`. Which operation gives `{3}`?

- ○ Union (`A | B`)
- ○ Intersection (`A & B`)
- ○ Difference (`A - B`)
- ○ Symmetric difference (`A ^ B`)

Check answer

7. Updating a set using set operations

These methods modify the original set rather than creating a new result.

```
A = {1, 2, 3}
B = {3, 4, 5}

A.update(B)
print(A)
```

`A` now contains `{1, 2, 3, 4, 5}`.

Other methods include:

```
A = {1, 2, 3}
B = {3, 4, 5}

A.intersection_update(B)
print(A)  # {3}
```

```
A = {1, 2, 3}
B = {3, 4, 5}

A.difference_update(B)
print(A)  # {1, 2}
```

```
A = {1, 2, 3}
B = {3, 4, 5}

A.symmetric_difference_update(B)
print(A)  # {1, 2, 4, 5}
```

The difference between a regular operation and its updating version:

- `A.union(B)` returns a new set.
- `A.update(B)` changes `A` itself.

8. Set comparison methods

`issubset()` — is every element contained in another set?

```
A = {1, 2}
B = {1, 2, 3, 4}

print(A.issubset(B))  # True
print(A <= B)         # True
```

`issuperset()` — does a set contain every element of another set?

```
print(B.issuperset(A))  # True
print(B >= A)           # True
```

`isdisjoint()` — do two sets have no common elements?

```
A = {1, 2}
B = {3, 4}

print(A.isdisjoint(B))  # True
```

If the sets share even one element, `isdisjoint()` returns `False`.

Proper subset and proper superset

A proper subset contains fewer elements than its proper superset.

```
A = {1, 2}
B = {1, 2, 3}

print(A < B)  # True
print(B > A)  # True
```

For ordinary subset checking, use `<=`; for proper subset checking, use `<`.

9. Set comprehension

Set comprehension is a concise way to create a set using an expression and a loop.

Syntax:

```
{expression for item in iterable if condition}
```

- `expression`: value to insert into the set.

- `item`: each element being processed.
- `iterable`: list, range, tuple, or another iterable.
- `condition`: optional filter.

Example 1: Squares of numbers

```
squares = {x ** 2 for x in range(1, 6)}

print(squares)
```

Result: `{1, 4, 9, 16, 25}`

Dry run

`x`	`x ** 2`	Added?
1	1	Yes
2	4	Yes
3	9	Yes
4	16	Yes
5	25	Yes

Example 2: Even numbers

```
even_numbers = {x for x in range(1, 11) if x % 2 == 0}

print(even_numbers)
```

Result: `{2, 4, 6, 8, 10}`

Example 3: Unique first letters

```
names = ["Aamir", "Aman", "Ravi", "Rahul"]

letters = {name[0] for name in names}

print(letters)
```

Result: `{'A', 'R'}`

Notice that repeated first letters are stored only once.

Example 4: Remove duplicates and square the values

```
numbers = [1, 2, 2, 3, 3, 4]

result = {n ** 2 for n in numbers}

print(result)
```

Result: `{1, 4, 9, 16}`

Common mistake: `{x for x in range(5)}` is a set comprehension, but `(x for x in range(5))` is a generator expression.

10. Converting between sets and other data types

List to set

```
numbers = [1, 2, 2, 3, 3, 4]
unique = set(numbers)

print(unique)
```

Useful for removing duplicates from a list.

Set to list

```
numbers = {10, 20, 30}
result = list(numbers)

print(result)
```

The resulting list's order is not guaranteed to match any intended ordering.

Set to sorted list

```
numbers = {40, 10, 30, 20}

result = sorted(numbers)

print(result)
```

Output:

```
[10, 20, 30, 40]
```

Important: `sorted()` returns a **list**, not a set.

String to set

```
word = "banana"

print(set(word))
```

Result: `{'b', 'a', 'n'}` in an unspecified display order.

Tuple to set

```
data = (1, 2, 2, 3)
print(set(data))
```

Result: `{1, 2, 3}`.

11. Mutable vs immutable objects inside sets

This is an important advanced Python concept.

Set elements must be **hashable**, meaning their hash value must remain stable while they are used as set elements.

Data type	Can be a set element?	Example
`int`	Yes	`{10}`
`float`	Yes	`{3.14}`

`str`	Yes	`{"Python"}`
`bool`	Yes	`{True}`
`tuple`	Yes, if all contents are hashable	`{(1, 2)}`
`list`	No	`{[1, 2]}` raises `TypeError`
`dict`	No	`{{"a": 1}}` raises `TypeError`
`set`	No	`{{1, 2}}` raises `TypeError`

Why can't a list be a set element?

Lists are mutable: their contents can change. This makes them unsuitable as set elements.

```
# This raises TypeError
# numbers = {[1, 2], [3, 4]}
```

If you need immutable equivalents, use tuples when their contents are hashable:

```
coordinates = {(10, 20), (30, 40)}

print(coordinates)
```

Can a tuple contain a list?

Yes, a tuple can contain a list, but that tuple is not hashable and cannot be added to a set.

```
value = (1, [2, 3])
```

```
# This raises TypeError
# s = {value}
```

Interview answer: An immutable outer object is not automatically hashable. Its contents also matter.

12. `set` vs `frozenset`

A `frozenset` is an immutable version of a set.

```
normal_set = {1, 2, 3}
fixed_set = frozenset([1, 2, 3])
```

You can perform operations such as union and intersection on both, but you cannot add or remove elements from a `frozenset`.

```
a = frozenset([1, 2, 3])
b = frozenset([3, 4, 5])

print(a | b)
print(a & b)
```

However, this is invalid:

```
# a.add(4)  # AttributeError
```

Comparison

Feature	`set`	`frozenset`
Mutable	Yes	No

Add/remove elements	Yes	No
Set operations	Yes	Yes
Can be a set element	No	Yes, if hashable
Can be a dictionary key	No	Yes, if hashable

Example of a `frozenset` inside a set:

```
groups = {frozenset([1, 2]), frozenset([3, 4])}

print(len(groups))
```

Output: `2`.

13. Duplicate values and equality

Python sets treat values that compare equal as the same element.

```
data = {1, 1, 2, 2, 3}
print(len(data))  # 3
```

An interesting example:

```
data = {True, 1, 1.0}
print(len(data))  # 1
```

Why? Because in Python, `True == 1 == 1.0` evaluates to `True`, and these values have compatible hashes.

Also remember that the empty set is false in a Boolean context:

```
s = set()

if not s:
    print("Set is empty")
```

14. Practical applications of sets

A. Remove duplicates while retaining the original list's order

A plain set does not guarantee the original list order. If you need to preserve it, use `dict.fromkeys()`:

```
numbers = [3, 1, 3, 2, 1, 4]

unique = list(dict.fromkeys(numbers))

print(unique)
```

Output:

```
[3, 1, 2, 4]
```

B. Find common students

```
python_students = {"Aamir", "Ravi", "Priya"}
java_students = {"Ravi", "Priya", "Aman"}

print(python_students & java_students)
```

Result: the students enrolled in both courses.

C. Find missing required skills

```
required = {"Python", "SQL", "Pandas", "NumPy"}
candidate = {"Python", "SQL", "NumPy"}

missing = required - candidate

print(missing)
```

Result: `{'Pandas'}`.

D. Find duplicate values in a list

```
numbers = [1, 2, 3, 2, 4, 1, 5]

seen = set()
duplicates = set()

for n in numbers:
    if n in seen:
        duplicates.add(n)
    else:
        seen.add(n)

print(duplicates)
```

Result: `{1, 2}`.

E. Check whether two lists contain the same unique values

```
a = [1, 2, 2, 3]
b = [3, 1, 2]

print(set(a) == set(b))
```

Output: `True`.

Be careful: this ignores both order and the number of occurrences. If frequency matters, compare the lists using a different method.

15. Advanced coding interview questions with answers and dry runs

These are practical questions you should be comfortable solving without looking at notes.

Question 1: Remove duplicates from a list

```
numbers = [10, 20, 10, 30, 20, 40]

result = list(set(numbers))

print(result)
```

Explanation: Convert the list to a set to remove duplicates, then convert it back to a list.

Note: the output order is not guaranteed. Use `list(dict.fromkeys(numbers))` if original order must be preserved.

Question 2: Find common elements in two lists

```
a = [1, 2, 3, 4]
b = [3, 4, 5, 6]

common = set(a) & set(b)

print(common)
```

Result: `{3, 4}`.

Dry run:

Element from A	In B?	Action
1	No	Skip
2	No	Skip
3	Yes	Include
4	Yes	Include

Question 3: Find elements that appear in only one list

```
a = [1, 2, 3, 4]
b = [3, 4, 5, 6]

result = set(a) ^ set(b)

print(result)
```

Result: `{1, 2, 5, 6}`.

Question 4: Check whether a list contains duplicates

```
numbers = [1, 2, 3, 4, 2]

if len(numbers) != len(set(numbers)):
    print("Duplicates found")
else:
    print("No duplicates")
```

Output: `Duplicates found`.

Logic: If converting the list to a set reduces its length, duplicates were present.

Question 5: Find the largest and second-largest unique numbers

```
numbers = [10, 20, 30, 20, 40, 40]

unique = sorted(set(numbers))

if len(unique) >= 2:
    print("Largest:", unique[-1])
    print("Second largest:", unique[-2])
else:
    print("Not enough unique values")
```

Output:

```
Largest: 40
Second largest: 30
```

Dry run:

1. Original list: `[10, 20, 30, 20, 40, 40]`
2. Remove duplicates: `{10, 20, 30, 40}`
3. Sort: `[10, 20, 30, 40]`
4. Last element is `40`; second-last is `30`.

Question 6: Find missing numbers from a range

```
numbers = [1, 2, 4, 6, 7]
expected = set(range(1, 8))

missing = expected - set(numbers)

print(sorted(missing))
```

Output:

```
[3, 5]
```

Question 7: Check whether two sets are disjoint

```
a = {1, 2, 3}
b = {4, 5, 6}

if a.isdisjoint(b):
```

```
    print("No common elements")
else:
    print("Common elements exist")
```

Output: `No common elements`.

Question 8: Find duplicate characters in a string

```
text = "programming"

seen = set()
duplicates = set()

for char in text:
    if char in seen:
        duplicates.add(char)
    else:
        seen.add(char)

print(duplicates)
```

Result: `{'r', 'g', 'm'}` in any order.

Question 9: Find elements common to three lists

```
a = [1, 2, 3, 4]
b = [2, 3, 4, 5]
c = [3, 4, 5, 6]

result = set(a) & set(b) & set(c)
```

```
print(result)
```

Output: `{3, 4}`.

Question 10: Check whether one list is a subset of another

```
required = [1, 2]
available = [1, 2, 3, 4]

print(set(required).issubset(set(available)))
```

Output: `True`.

This checks membership, not the number of occurrences. If repeated elements matter, sets alone are insufficient.

16. Set time complexity

Python sets are implemented using hash tables. Their operations are generally efficient on average.

Operation	Average time complexity
Add an element	$O(1)$
Check membership (`x in s`)	$O(1)$
Remove an element	$O(1)$
Discard an element	$O(1)$
Copy a set	$O(n)$
Iterate through a set	$O(n)$

Union of sets	$O(n + m)$, typically
Intersection	Depends on the set sizes; typically $O(\min(n, m))$ when implemented by iterating the smaller set and probing the larger
Sort a set using `sorted()`	$O(n \log n)$

Here, n and m are the numbers of elements in the sets.

These are average-case expectations, not unconditional guarantees. Hash collisions and implementation details can affect performance.

Why are sets fast for membership checking?

Consider checking whether a value exists in a list versus a set:

```
numbers_list = list(range(100000))
numbers_set = set(numbers_list)

print(99999 in numbers_list)
print(99999 in numbers_set)
```

A list may need to examine many elements one by one. A set generally uses hashing to locate a candidate quickly.

17. Common mistakes to avoid

Mistake	Correct approach
`s = {}` to create a set	Use `s = set()`
`s[0]` to access an element	Use membership testing or iteration
Assuming sets preserve insertion order	Use a list or ordered deduplication technique

Adding a list with `s.add([1, 2])`	Use `s.update([1, 2])` to add its elements
Using `remove()` when the element might be missing	Use `discard()` or check membership
Expecting `sorted(s)` to return a set	It returns a list
Expecting `set()` to preserve duplicates	Sets eliminate duplicates
Putting a dictionary or list inside a set	Use a hashable representation when appropriate
Using sets when duplicate counts matter	Use a list or `collections.Counter`
Modifying a set while iterating over it	Iterate over a copy or build a new set

Example of safely removing elements while iterating:

```
numbers = {1, 2, 3, 4, 5}

for n in numbers.copy():
    if n % 2 == 0:
        numbers.remove(n)

print(numbers)
```

Output: `{1, 3, 5}`.

18. Most asked Python set interview questions

Here are short answers to revise before interviews.

1. **What is a set?** A mutable collection of unique, hashable elements.
2. **Are sets ordered?** No guaranteed positional order.
3. **Can sets contain duplicates?** No.
4. **Can sets contain lists?** No, because lists are unhashable.

5. **How do you create an empty set?** `set()`.
6. **What is the difference between** `add()` **and** `update()`**?** `add()` inserts one element; `update()` inserts elements from iterables.
7. **What is the difference between** `remove()` **and** `discard()`**?** `remove()` raises `KeyError` when missing; `discard()` does not.
8. **What does** `pop()` **do?** Removes and returns an arbitrary element.
9. **What is union?** All unique elements from either set.
10. **What is intersection?** Elements shared by both sets.
11. **What is difference?** Elements in one set but not the other.
12. **What is symmetric difference?** Elements in exactly one of two sets.
13. **What is set comprehension?** A concise syntax for constructing a set from an iterable.
14. **What is a** `frozenset`**?** An immutable set.
15. **Can a tuple be a set element?** Yes, if all its contents are hashable.
16. **Can a set be a dictionary key?** No; a `frozenset` can be.
17. **Can a set contain both** `True` **and** `1` **as separate elements?** No, because they compare equal and have compatible hashes.
18. **What is the average membership-checking complexity?** $O(1)$.
19. **How do you remove duplicates from a list?** Convert it to a set, if order does not matter.
20. **How do you preserve list order while removing duplicates?** `list(dict.fromkeys(my_list))`, for hashable values.

19. Practice questions — test your knowledge

Try solving these yourself before opening the answers.

1 Create a set containing 1, 2, 3, 4, 5.

Reveal answer

2 Remove duplicates from [1, 2, 2, 3, 4, 4].

Reveal answer

3 Add 50 to a set called numbers.

Reveal answer

4 Find the union of A and B.

Reveal answer

5 Find common elements between A and B.

Reveal answer

6 Find elements in A but not B.

Reveal answer

7 Create a set of squares from 1 to 10.

Reveal answer

8 Check whether 25 exists in numbers.

Reveal answer

9 Find missing numbers between 1 and 10.

Reveal answer

10 Create an immutable set.

Reveal answer

Show all answers

20. Final revision mind map

PYTHON SET

Unique, hashable elements

Creation

`{1, 2, 3}`

`set()`

`frozenset()`

Modification

`add()` / `update()`

`remove()` / `discard()`

`pop()` / `clear()`

Operations

Union: `|`

Intersection: `&`

Difference: `-`

Symmetric: `^`

Advanced

Comprehensions

Subset / superset

Hashability

Time complexity

21. The 5 rules you must remember

1. Use `set()` to create an empty set.
2. Sets automatically eliminate duplicate elements.
3. Set elements must be hashable.
4. Use `|`, `&`, `-`, and `^` for the four fundamental set operations.

5. Use `frozenset()` when you need an immutable set.

Recommended next step: Practise these set concepts until you can write the 10 coding exercises without help, then study Python dictionaries in the same depth. Dictionaries and sets are closely related because both use hashing internally.

You said:

Complete code and concept explain basics to advanced set for AIML

ChatGPT said:

Python Set for AI/ML — Complete Guide from Basics to Advanced

This guide is designed for your **Python, Data Science, and AI/ML interview preparation**. You will learn not only set syntax, but also how sets help with data cleaning, feature engineering, categorical data, text processing, and dataset comparison.

By the end, you should be able to explain set concepts, write interview programs, dry-run code, and use sets in practical AI/ML workflows.

1. What is a set in Python?

A **set** is a built-in data type that stores unique, hashable elements.

```
numbers = {10, 20, 30, 10, 20}

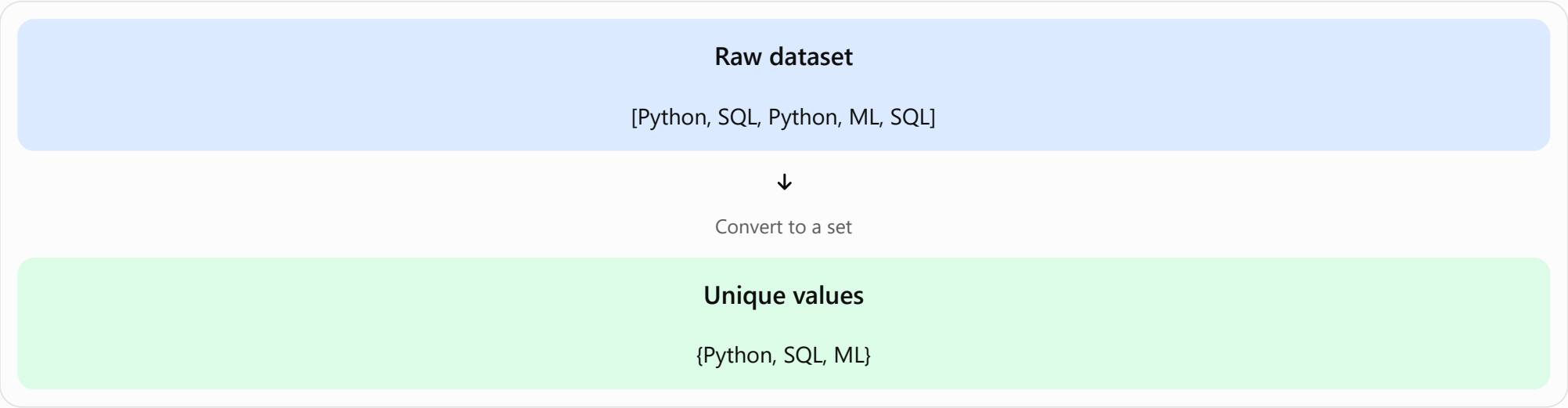
print(numbers)
print(len(numbers))
```

Output:

```
{10, 20, 30}
3
```

Duplicates are removed automatically. The order shown in the output is not guaranteed.

Visual understanding



Important properties

Property	Explanation
Unique	Duplicate values are eliminated
Unordered	No guaranteed positional order
Mutable	Elements can be added or removed
Unindexed	`s[0]` is not supported
Hashable elements	Every element must be hashable
Efficient membership	Average-case `O(1)` lookup

2. Creating sets

Method 1: Curly braces

```
numbers = {1, 2, 3, 4, 5}
skills = {"Python", "SQL", "Machine Learning"}

print(numbers)
print(skills)
```

Method 2: Using `set()`

```
numbers = set([1, 2, 2, 3, 4])
print(numbers)
```

Output: `{1, 2, 3, 4}`

Method 3: Empty set

```
a = set()
b = {}

print(type(a))  # <class 'set'>
print(type(b))  # <class 'dict'>
```

Interview question: Why does `{}` create a dictionary rather than a set?

Because Python uses `{}` for an empty dictionary. Use `set()` to create an empty set.

Method 4: Convert strings, tuples, and lists

```
print(set("banana"))
print(set((1, 2, 2, 3)))
print(set(["AI", "ML", "AI"]))
```

Each result contains unique elements.

3. Accessing and checking elements

Sets do not support indexing or slicing.

```
skills = {"Python", "SQL", "ML"}

print("Python" in skills)
print("Java" not in skills)
```

Output:

```
True
True
```

Loop through a set:

```
for skill in skills:
    print(skill)
```

The order of iteration is not guaranteed.

AI/ML example: Check required skills

```
required_skills = {"Python", "NumPy", "Pandas"}
my_skills = {"Python", "SQL", "NumPy", "Pandas"}

for skill in required_skills:
    if skill in my_skills:
        print(skill, "is available")
```

This demonstrates membership testing. In real ML workflows, sets can help compare feature names, required columns, or supported categories.

4. Adding elements to a set

`add()` — add one element

```
features = {"age", "salary"}

features.add("experience")
features.add("age")  # Duplicate, so no new element

print(features)
```

The set contains `age`, `salary`, and `experience`.

`update()` — add multiple elements

```
features = {"age", "salary"}

features.update(["experience", "education"])
features.update(("city", "department"))
```

```
print(features)
```

Dry run

Step	Operation	Number of unique elements
1	`{"age", "salary"}`	2
2	Add `"experience"`	3
3	Add `"age"` again	3
4	Update with `"education"`	4

Remember:

- `add()` inserts the supplied object as one element.
- `update()` inserts elements from the supplied iterable.
- Both methods return `None`.

```
s = {1, 2}
s.add(3)
s.update([4, 5])
print(s)
```

5. Removing elements

Python provides four important methods.

```
numbers = {10, 20, 30, 40}
```

```
numbers.remove(20)   # Error if 20 is missing
numbers.discard(100) # Safe if 100 is missing

removed = numbers.pop()  # Removes an arbitrary element
print("Removed:", removed)

numbers.clear()
print(numbers)
```

Method	Purpose	Missing element
`remove(x)`	Remove `x`	Raises `KeyError`
`discard(x)`	Remove `x` if present	No error
`pop()`	Remove and return an arbitrary element	Raises `KeyError` if empty
`clear()`	Remove all elements	No error

AI/ML use case: You might remove an unwanted feature name from a set of candidate columns, or use set differences to identify columns that should be excluded.

---

6. Four fundamental set operations

This is one of the most important sections for interviews and real data work.

Suppose two datasets contain these feature names:

```
dataset_A = {"age", "salary", "experience", "education"}
dataset_B = {"age", "salary", "city", "education"}
```

Visualize the two datasets

Dataset A	Dataset B
age	age
salary	salary
experience	city
education	education

Common features: age, salary, education. Different features: experience and city.

6.1 Union (`|`): combine unique elements

```
print(dataset_A | dataset_B)
print(dataset_A.union(dataset_B))
```

Result contains `age`, `salary`, `experience`, `education`, and `city`.

Use union when combining feature names, category vocabularies, or unique labels from different sources.

6.2 Intersection (`&`): find common elements

```
print(dataset_A & dataset_B)
print(dataset_A.intersection(dataset_B))
```

Result:

```
{"age", "salary", "education"}
```

Use intersection to find shared features, common categories, or common IDs across datasets.

6.3 Difference (`-`): find elements missing from the other set

```
print(dataset_A - dataset_B)
```

Result:

```
{"experience"}
```

Reverse the order:

```
print(dataset_B - dataset_A)
```

Result:

```
{"city"}
```

This is useful for detecting missing columns and comparing feature schemas.

6.4 Symmetric difference (`^`): find elements exclusive to either set

```
print(dataset_A ^ dataset_B)
```

Result:

```
{"experience", "city"}
```

These features occur in exactly one of the datasets.

Quick comparison

Operation	Symbol	AI/ML example
Union	`A | B`	Combine unique categories
Intersection	`A & B`	Find shared features
Difference	`A - B`	Find missing columns
Symmetric difference	`A ^ B`	Find schema mismatches

Interactive quiz

You have `train_columns = {"age", "salary", "city"}` and `test_columns = {"age", "salary", "education"}`.

Which expression finds columns missing from the test dataset but present in the training dataset?

- `train_columns | test_columns`
- `train_columns & test_columns`
- `train_columns - test_columns`
- `train_columns ^ test_columns`

Check answer

7. Set update operations

Normal operations return a new set. Update operations modify the original set.

```
A = {1, 2, 3}
B = {3, 4, 5}

A.update(B)
print(A)
```

Output: `{1, 2, 3, 4, 5}`

Other methods:

```
A = {1, 2, 3}
A.intersection_update({2, 3, 4})
print(A)  # {2, 3}
```

```
A = {1, 2, 3}
A.difference_update({2, 3})
print(A)  # {1}
```

```
A = {1, 2, 3}
A.symmetric_difference_update({3, 4})
print(A)  # {1, 2, 4}
```

Use the updating variants when you intentionally want to modify an existing set.

8. Subset, superset, and disjoint sets

These methods help check relationships between groups.

`issubset()` — `A` is contained in `B`

```
A = {"Python", "NumPy"}
B = {"Python", "NumPy", "Pandas", "Scikit-learn"}

print(A.issubset(B))  # True
print(A <= B)         # True
```

`issuperset()` — `A` contains all elements of `B`

```
print(B.issuperset(A))  # True
print(B >= A)           # True
```

`isdisjoint()` — no shared elements

```
A = {"Python", "SQL"}
B = {"Java", "C++"}

print(A.isdisjoint(B))  # True
```

Proper subset

```
A = {1, 2}
B = {1, 2, 3}
```

```
print(A < B)  # True
print(A <= B) # True
```

`A < B` means that `A` is a proper subset of `B`; the sets must not be equal. `A <= B` permits equality.

AI/ML example: Check whether a dataset contains every required input feature.

```
required = {"age", "income", "experience"}
actual = {"age", "income", "experience", "city"}

if required.issubset(actual):
    print("All required features are present")
else:
    print("Required features are missing")
```

---

9. Set comprehension — advanced creation technique

Set comprehension creates a set using a compact expression.

Syntax:

```
{expression for item in iterable if condition}
```

Example 1: Squares

```
squares = {x ** 2 for x in range(1, 6)}
print(squares)
```

Result: `{1, 4, 9, 16, 25}`

Example 2: Filter even numbers

```
evens = {x for x in range(1, 21) if x % 2 == 0}
print(evens)
```

Result: the even numbers from 2 through 20.

Example 3: Unique categories in a dataset

```
cities = ["Delhi", "Mumbai", "Delhi", "Jaipur", "Mumbai"]

unique_cities = {city for city in cities}

print(unique_cities)
```

Result: `{"Delhi", "Mumbai", "Jaipur"}` in no guaranteed order.

Example 4: Extract unique characters

```
text = "machinelearning"

unique_chars = {char for char in text if char.isalpha()}

print(unique_chars)
```

This is useful for exploring unique characters in text-processing tasks.

Example 5: Normalize categories

```
labels = ["Cat", "DOG", "cat", "Dog", "bird"]

normalized = {label.lower() for label in labels}

print(normalized)
```

Result: `{"cat", "dog", "bird"}`.

Normalization is often part of data cleaning, although actual categorical data may require additional handling of missing values and inconsistent spellings.

10. Hashability and mutable versus immutable objects

This concept is important for Python interviews and understanding how sets work internally.

A **hashable object** has a hash value that remains stable during its lifetime and can be compared for equality. Set elements must be hashable.

Which data types can be set elements?

Data type	Allowed?	Reason
Integer	Yes	Hashable
Float	Yes	Hashable
String	Yes	Hashable
Boolean	Yes	Hashable
Tuple	Sometimes	All elements must be hashable
List	No	Unhashable

Dictionary	No	Unhashable
Mutable set	No	Unhashable
`frozenset`	Yes	Hashable if its contents are hashable

Example: Why a list fails

```
# Invalid: a list is unhashable
# data = {[1, 2], [3, 4]}
```

Python raises `TypeError`.

Use tuples instead:

```
coordinates = {(10, 20), (30, 40)}
print(coordinates)
```

An important tuple exception

```
valid = (1, 2, 3)
invalid = (1, [2, 3])

a = {valid}  # Works

# b = {invalid}  # TypeError
```

A tuple is immutable, but that does not guarantee it is hashable. Its elements must also be hashable.

Boolean and integer equality

```
values = {True, 1, 1.0}
print(len(values))
```

Output:

```
1
```

`True`, `1`, and `1.0` compare equal and have compatible hashes, so they cannot coexist as separate set elements.

11. `set` versus `frozenset`

A `frozenset` is an immutable set. Once created, its elements cannot be added or removed.

```
A = {1, 2, 3}
B = frozenset([1, 2, 3])

print(type(A))
print(type(B))
```

Output:

```
<class 'set'>
<class 'frozenset'>
```

Both support operations such as union and intersection.

```
A = frozenset([1, 2, 3])
B = frozenset([3, 4, 5])

print(A | B)
print(A & B)
```

But this is invalid:

```
# A.add(10)  # AttributeError
```

Advanced example: Set of sets

Ordinary sets cannot be nested directly because they are unhashable.

```
# Invalid:
# groups = {{1, 2}, {3, 4}}
```

Use `frozenset`:

```
groups = {
    frozenset([1, 2]),
    frozenset([3, 4])
}

print(len(groups))  # 2
```

This can be useful when you need immutable collections as dictionary keys or elements of another set.

12. Copying sets

Shallow copy

```
A = {1, 2, 3}
B = A.copy()

B.add(4)

print(A)  # {1, 2, 3}
print(B)  # {1, 2, 3, 4}
```

Changing `B` does not modify `A`.

Assignment is different from copying

```
A = {1, 2, 3}
B = A

B.add(4)

print(A)  # {1, 2, 3, 4}
print(B)  # {1, 2, 3, 4}
```

`B = A` makes both names refer to the same set object.

This distinction matters when preparing data structures for processing.

---

13. Built-in functions that work with sets

```
numbers = {10, 20, 30, 40}

print(len(numbers))  # 4
print(min(numbers))  # 10
print(max(numbers))  # 40
print(sum(numbers))  # 100
print(sorted(numbers))  # [10, 20, 30, 40]
```

Function	Purpose
`len(s)`	Count unique elements
`min(s)`	Find smallest comparable element
`max(s)`	Find largest comparable element
`sum(s)`	Add numeric elements
`sorted(s)`	Return a sorted list
`any(s)`	Check whether any element is truthy
`all(s)`	Check whether all elements are truthy
`enumerate(s)`	Iterate with counter values

Example:

```
scores = {0, 1, 0, 1}

print(any(scores))  # True
print(all(scores))  # False
```

Be careful with `sum()`, `min()`, and `max()`: they are meaningful only for suitable element types.

14. Advanced coding questions for interviews

Program 1: Find duplicate values in a list

```
numbers = [10, 20, 30, 20, 40, 10, 50]

seen = set()
duplicates = set()

for number in numbers:
    if number in seen:
        duplicates.add(number)
    else:
        seen.add(number)

print("Duplicates:", duplicates)
```

Output:

```
Duplicates: {10, 20}
```

Dry run:

Current value	Already seen?	Duplicate set
10	No	`{}`

20	No	`{}`
30	No	`{}`
20	Yes	`{20}`
40	No	`{20}`
10	Yes	`{10, 20}`
50	No	`{10, 20}`

Program 2: Find the second-largest unique number

```
numbers = [10, 40, 20, 40, 30, 20]

unique_numbers = sorted(set(numbers))

if len(unique_numbers) >= 2:
    print(unique_numbers[-2])
else:
    print("Not enough unique numbers")
```

Output:

```
30
```

Why? The unique sorted values are `[10, 20, 30, 40]`. The second-last element is `30`.

Program 3: Find missing numbers in a sequence

```
numbers = [1, 2, 4, 6, 7, 10]

expected = set(range(1, 11))
actual = set(numbers)

missing = expected - actual

print(sorted(missing))
```

Output:

```
[3, 5, 8, 9]
```

This pattern is useful for checking missing IDs or incomplete integer sequences.

Program 4: Find common elements in three lists

```
a = [1, 2, 3, 4, 5]
b = [2, 3, 4, 6]
c = [3, 4, 7]

common = set(a) & set(b) & set(c)

print(common)
```

Output:

```
{3, 4}
```

Program 5: Check whether two lists contain the same unique elements

```
a = [1, 2, 2, 3]
b = [3, 1, 2]

if set(a) == set(b):
    print("Same unique elements")
else:
    print("Different unique elements")
```

Output:

```
Same unique elements
```

Important: this does not check duplicate counts. `[1, 1, 2]` and `[1, 2]` produce equal sets.

Program 6: Find duplicate characters

```
text = "mississippi"

seen = set()
duplicates = set()

for char in text:
    if char in seen:
        duplicates.add(char)
    else:
```

```
        seen.add(char)

print(duplicates)
```

Result: `{'i', 's', 'p'}` in any order.

Program 7: Remove duplicates while preserving order

```
numbers = [4, 2, 4, 1, 2, 5, 1]

result = list(dict.fromkeys(numbers))

print(result)
```

Output:

```
[4, 2, 1, 5]
```

Why not just use `list(set(numbers))` ? Because a set does not guarantee the original order.

Program 8: Find all unique words in a sentence

```
sentence = "Python helps AI and Python helps ML"

words = sentence.lower().split()
unique_words = set(words)

print(unique_words)
print("Unique word count:", len(unique_words))
```

This is a basic text-analysis task. Real natural-language processing may additionally require punctuation removal, tokenization, and normalization.

---

15. Applying sets to AI/ML and Data Science

This is where the concept becomes especially useful for your AI/ML preparation.

Use case 1: Data cleaning — identify unique categories

Imagine a dataset has a `city` column.

```
cities = [
    "Delhi", "Mumbai", "Delhi",
    "Jaipur", "Mumbai", "Patna"
]

unique_cities = set(cities)

print(unique_cities)
print("Number of unique cities:", len(unique_cities))
```

You can use this to inspect the number of unique categorical values before encoding them.

However, sets remove duplicates and discard ordering. Do not replace a DataFrame column with a set if row-level records must be preserved.

Use case 2: Feature engineering — compare features

```
train_features = {
    "age", "income", "education", "experience"
}
```

```
test_features = {
    "age", "income", "education", "city"
}

missing_in_test = train_features - test_features
extra_in_test = test_features - train_features
common_features = train_features & test_features

print("Missing:", missing_in_test)
print("Extra:", extra_in_test)
print("Common:", common_features)
```

Expected results:

```
Missing: {'experience'}
Extra: {'city'}
Common: {'age', 'income', 'education'}
```

This helps identify mismatched feature schemas before passing data into a trained model. Real pipelines should also validate column types, order, and preprocessing requirements.

Use case 3: Text processing — vocabulary exploration

```
documents = [
    "machine learning",
    "deep learning",
    "machine intelligence"
]

vocabulary = set()
```

```
for document in documents:
    for word in document.lower().split():
        vocabulary.add(word)

print(vocabulary)
```

Unique vocabulary words:

```
machine, learning, deep, intelligence
```

Sets can help explore unique tokens. However, actual text vectorizers usually maintain an explicit vocabulary-to-index mapping rather than relying on set iteration order.

Use case 4: Compare class labels

```
train_labels = {"cat", "dog", "bird"}
test_labels = {"cat", "dog", "horse"}

unknown_labels = test_labels - train_labels

print(unknown_labels)
```

Output:

```
{'horse'}
```

This identifies a class label appearing in test data that was not present in the training labels. Depending on the model and task, this may require special handling.

Use case 5: Dataset validation

```
required_columns = {
    "age", "income", "education", "target"
}

actual_columns = {
    "age", "income", "education", "target", "city"
}

missing = required_columns - actual_columns
extra = actual_columns - required_columns

if not missing:
    print("Required columns are available")
else:
    print("Missing columns:", missing)

print("Extra columns:", extra)
```

This can be part of a data-validation step before training.

Important ML rule: Do not construct preprocessing rules using test-set information when that would leak information into model training. Use the training data to fit preprocessing steps, then apply the fitted transformations to validation and test data.

16. Sets with NumPy and Pandas

Sets work alongside popular Data Science libraries, but they are not replacements for NumPy arrays or Pandas Series.

A. NumPy: find unique values

```
import numpy as np

arr = np.array([10, 20, 10, 30, 20])

unique_values = np.unique(arr)

print(unique_values)
```

Output:

```
[10 20 30]
```

`np.unique()` is often preferable when working directly with NumPy arrays because it returns a NumPy array.

B. Pandas: find unique categories

```
import pandas as pd

df = pd.DataFrame({
    "city": ["Delhi", "Mumbai", "Delhi", "Jaipur"],
    "score": [80, 90, 85, 88]
})

print(df["city"].unique())
print(df["city"].nunique())
```

`unique()` returns the distinct values in their order of appearance. `nunique()` returns the number of unique values, excluding missing values by default.

You can also use a set:

```
city_set = set(df["city"].dropna())

print(city_set)
```

Use `.dropna()` here to avoid including missing values in the set. In Pandas, missing values require special handling and may not behave like ordinary values.

C. Compare columns between DataFrames

```
import pandas as pd

train_df = pd.DataFrame({
    "age": [20, 25],
    "income": [30000, 50000],
    "city": ["Delhi", "Jaipur"]
})

test_df = pd.DataFrame({
    "age": [22, 28],
    "income": [35000, 60000],
    "experience": [1, 3]
})

train_columns = set(train_df.columns)
test_columns = set(test_df.columns)

print("Common:", train_columns & test_columns)
print("Train only:", train_columns - test_columns)
print("Test only:", test_columns - train_columns)
```

This is a practical schema-validation technique.

17. Complete AI/ML mini-project using sets

Let's combine several concepts into a small, realistic data-preparation exercise.

Goal: Inspect a dataset, identify duplicate categories, compare training and test features, and validate required columns.

Mini-project: Dataset quality checker

Python fundamentals · Data validation · ML preparation

Input

Training columns, test columns, city categories, and required model features.

Output

Unique categories, missing features, extra features, and validation status.

Complete runnable code

```
# ========================================
# AI/ML MINI-PROJECT: DATASET QUALITY CHECKER
# ========================================

# 1. Sample dataset information
train_columns = {
    "age",
    "income",
    "education",
    "experience",
    "target"
```

```
}

test_columns = {
    "age",
    "income",
    "education",
    "city"
}

cities = [
    "Delhi",
    "Mumbai",
    "Delhi",
    "Jaipur",
    "Mumbai",
    "Patna"
]

required_features = {
    "age",
    "income",
    "education",
    "experience"
}

# 2. Find unique city categories
unique_cities = set(cities)

print("Unique cities:", unique_cities)
print("Number of unique cities:", len(unique_cities))

# 3. Find shared and different columns
common_columns = train_columns & test_columns
missing_in_test = train_columns - test_columns
```

```
extra_in_test = test_columns - train_columns

print("\nCommon columns:", common_columns)
print("Missing in test:", missing_in_test)
print("Extra in test:", extra_in_test)

# 4. Validate required features
available_features = test_columns

missing_required = required_features - available_features

if missing_required:
    print("\nValidation failed!")
    print("Required features missing:", missing_required)
else:
    print("\nAll required features are available.")

# 5. Check whether test features are a subset
# of training features, excluding target.
model_features = train_columns - {"target"}

unknown_features = test_columns - model_features

print("\nFeatures not in training inputs:", unknown_features)
```

Understand the dry run

Step	Operation	What it tells you
1	`set(cities)`	Removes duplicate city names
2	`train_columns & test_columns`	Finds common columns

3	`train_columns - test_columns`	Finds columns absent from test
4	`required_features - test_columns`	Finds missing required model features
5	`test_columns - model_features`	Finds test columns not known as model inputs

Expected logical findings:

- Unique cities: Delhi, Mumbai, Jaipur, Patna.
- Common columns: `age`, `income`, `education`.
- Training-only columns: `experience`, `target`.
- Test-only columns: `city`.
- Required test feature missing: `experience`.

Notice that `target` is deliberately excluded from model inputs because it is the prediction label, not an input feature.

This is a validation exercise, not a complete ML pipeline. Before model training, you would need to align feature columns and fit appropriate preprocessing steps on training data.

18. Time complexity of Python sets

Python sets use hash tables internally, making many operations efficient.

Operation	Average time complexity
Add an element	$O(1)$
Check membership	$O(1)$
Remove an element	$O(1)$
Copy a set	$O(n)$
Iterate through a set	$O(n)$

Union	$O(n+m)$
Sort using `sorted()`	$O(n \log n)$

Here, n and m represent the sizes of the input sets. These are typical average-case expectations; hash collisions can affect performance.

Why are sets useful for large datasets?

```
# Membership comparison
numbers_list = list(range(100_000))
numbers_set = set(numbers_list)

print(99_999 in numbers_list)
print(99_999 in numbers_set)
```

Both return `True`, but the list may need to scan many elements, whereas the set typically checks membership in average $O(1)$ time.

19. Common mistakes in AI/ML interviews

Mistake	Correct approach
Using `{}` for an empty set	Use `set()`
Assuming set order is fixed	Use a list if order matters
Using `s[0]`	Sets do not support indexing
Putting a list inside a set	Use a hashable representation
Using a set to count occurrences	Use `collections.Counter`
Using `list(set(data))` when order matters	Use `list(dict.fromkeys(data))` for hashable elements

Confusing `union()` with `update()`	Union returns a result; update modifies the set
Assuming a tuple is always hashable	Its contents must also be hashable
Comparing only sets to validate duplicated records	Check counts and row-level data separately
Fitting preprocessing on all data before splitting	Fit on training data to avoid data leakage

20. Final interview revision questions

Practise explaining these aloud in your own words.

1. What is a Python set?

2. Why is membership checking usually fast?

3. What is the difference between union and intersection?

4. How do you find missing columns in a dataset?

5. What is a frozenset?

6. Why can't a set contain a list?

7. How can sets help in NLP?

8. How can sets help validate an ML dataset?

9. Does converting a list to a set preserve duplicates or order?

10. What is the difference between a set and a NumPy array?

21. Your AI/ML Set mastery checklist

Study progress 0 of 10

- [ ] Create empty and non-empty sets
- [ ] Add, update, remove, and clear elements
- [ ] Use union, intersection, difference, and symmetric difference
- [ ] Understand subset, superset, and disjoint checks
- [ ] Write set comprehensions
- [ ] Understand hashability and frozenset
- [ ] Solve duplicate and missing-value coding questions
- [ ] Compare dataset columns using set operations
- [ ] Use sets alongside NumPy and Pandas
- [ ] Explain time complexity and common interview mistakes

Recommended learning order for AI/ML: Python sets → dictionaries → functions → NumPy → Pandas → exploratory data analysis → feature engineering → model training.

The most important practical skill is being able to use sets to identify **unique values, common elements, missing features, and differences between datasets** while understanding when a set is not the right data structure.